data_preprocessing.ipynb

Scale, Standardize, or Normalize with scikit-learn

Когда использовать MinMaxScaler, RobustScaler, StandardScaler, Normalizer

Зачем масштабировать, стандартизировать или нормализовать?

Многие алгоритмы работают лучше или сходятся быстрее, когда численные признаки имеют сопоставимые масштабы:

- линейная и логистическая регрессия с регуляризацией
- ближайшие соседи
- нейронные сети
- метод опорных векторов с RBF-ядром
- анализ главных компонентов

Масштабирование само по себе не делает распределение нормальным.

MinMaxScaler, ***RobustScaler***, ***StandardScaler***, а также ***Normalizer*** это методы предварительной обработки данных для машинного обучения. Какой метод вам нужен, зависит от типа вашей модели и значений данных.

Все эти методы находятся в scikit-learn в модуле `preprocessing` и имеют интерфейс `fit`, `transform`, `fit_transform`.

Начнём с небольшого самостоятельного примера. Сначала разделяем данные, затем обучаем преобразование только на `X_train`. Для `X_test` используем `transform`, без повторного обучения.

```
from sklearn import preprocessing
from sklearn.model_selection import train_test_split

X_example = [[1.0, 10.0], [2.0, 20.0], [3.0, 30.0], [4.0, 40.0]]
X_train, X_test = train_test_split(X_example, test_size=0.25, random_state=42)

mm_scaler = preprocessing.MinMaxScaler()
X_train_minmax = mm_scaler.fit_transform(X_train)
X_test_minmax = mm_scaler.transform(X_test)

print("Train после преобразования:", X_train_minmax)
print("Test после того же преобразования:", X_test_minmax)
```

Мы рассмотрим несколько распределений и применим к ним каждый из четырех методов scikit-learn.

```
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn import preprocessing
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
matplotlib.style.use('ggplot')
np.random.seed(42)

# Работает из папки ноутбука и из корня teacher-репозитория.
DATA_DIR = Path('.')
if not (DATA_DIR / 'train.csv').is_file():
    DATA_DIR = Path('lesson_03_features_and_preprocessing')
```

Original Distributions

Let's make several types of random distributions.

```
#create columns of various distributions
df = pd.DataFrame({
    'beta': np.random.beta(5, 1, 1000) * 60,        # beta
    'exponential': np.random.exponential(10, 1000), # exponential
    'normal_p': np.random.normal(10, 2, 1000),      # normal platykurtic
    'normal_l': np.random.normal(10, 10, 1000),     # normal leptokurtic
})

# make bimodal distribution
first_half = np.random.normal(20, 3, 500)
second_half = np.random.normal(-20, 3, 500)
bimodal = np.concatenate([first_half, second_half])

df['bimodal'] = bimodal

# create list of column names to use later
col_names = list(df.columns)
```

Let's plot our original distributions.

```
# plot original distribution plot
fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('Original Distributions')

sns.kdeplot(df['beta'], ax=ax1)
sns.kdeplot(df['exponential'], ax=ax1)
sns.kdeplot(df['normal_p'], ax=ax1)
sns.kdeplot(df['normal_l'], ax=ax1)
sns.kdeplot(df['bimodal'], ax=ax1);
```

```
df.head()
```

Let's see what are the means are.

```
df.mean()
```

If you'd like more summary statistics:

```
df.describe()
```

```
df.plot()
```

These values are all in the same ballpark.

Добавим признаки с огромными значениями

Может быть стоимость квартиры, машины и тд

```
normal_big = np.random.normal(1000000, 10000, (1000,1))  # normal distribution of large values
df['normal_big'] = normal_big
```

```
col_names.append('normal_big')
```

```
df['normal_big'].plot(kind='kde')
```

```
df.normal_big.mean()
```

Получили нормальное распределение со средним в районе 1,000,000.

Если выведем всё на одном графике - ничего не понятно

```
# plot original distribution plot with larger value feature
fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('Original Distributions')

sns.kdeplot(df['beta'], ax=ax1)
sns.kdeplot(df['exponential'], ax=ax1)
sns.kdeplot(df['normal_p'], ax=ax1)
sns.kdeplot(df['normal_l'], ax=ax1)
sns.kdeplot(df['bimodal'], ax=ax1);
sns.kdeplot(df['normal_big'], ax=ax1);
```

Распределение с огромным средним справа внизу. Теперь посмотрим на график значений:

```
df.plot()
```

```
df.describe()
```

А теперь посмотрим что будет, если мы используем scaling. Сначала заюзаем MinMax Scaler.

MinMaxScaler

Для каждого значения в объекте MinMaxScaler вычитает минимальное значение и затем делит на разницу между исходным максимумом и исходным минимумом.

```
# plot original distribution plot
fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('Original Distributions')

sns.kdeplot(df['beta'], ax=ax1)
sns.kdeplot(df['exponential'], ax=ax1)
sns.kdeplot(df['normal_p'], ax=ax1)
sns.kdeplot(df['normal_l'], ax=ax1)
sns.kdeplot(df['bimodal'], ax=ax1);
```

```
mm_scaler = preprocessing.MinMaxScaler()
df_mm = mm_scaler.fit_transform(df)

df_mm = pd.DataFrame(df_mm, columns=col_names)

fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('After MinMaxScaler')

sns.kdeplot(df_mm['beta'], ax=ax1)
sns.kdeplot(df_mm['exponential'], ax=ax1)
sns.kdeplot(df_mm['normal_p'], ax=ax1)
sns.kdeplot(df_mm['normal_l'], ax=ax1)
sns.kdeplot(df_mm['bimodal'], ax=ax1)
sns.kdeplot(df_mm['normal_big'], ax=ax1);
```

Notice how the shape of each distribution remains the same, but now the values are between 0 and 1.

```
df_mm['beta'].min()
```

```
df_mm['beta'].max()
```

Let's look at the minimums and maximums for each column prior to scaling.

```
df.describe()
```

```
df_mm.describe()
```

```
df["beta"].min(), df["beta"].max()
```

```
np.mean((df["beta"] - df["beta"].min()) / (df["beta"].max() - df["beta"].min()))
```

Обратим внимание на минимум и максимум для каждого признака после MinMaxScaler.

MinMaxScaler сохраняет форму распределения каждого отдельного признака при линейном преобразовании, но меняет вклад признаков в расстояния между объектами.

MinMaxScaler не уменьшает влияние выбросов: минимум и максимум сами зависят от них.

Диапазон по умолчанию — от 0 до 1 для данных, на которых выполнен `fit` . Новые значения вне обучающего диапазона могут преобразоваться в числа меньше 0 или больше 1.

RobustScaler

RobustScaler преобразует вектор признаков путем вычитания медианы, а затем деления на диапазон между четвертями (значение 75% - значение 25%).

```
# plot original distribution plot
fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('Original Distributions')

sns.kdeplot(df['beta'], ax=ax1)
sns.kdeplot(df['exponential'], ax=ax1)
sns.kdeplot(df['normal_p'], ax=ax1)
sns.kdeplot(df['normal_l'], ax=ax1)
sns.kdeplot(df['bimodal'], ax=ax1);
```

```
r_scaler = preprocessing.RobustScaler()
df_r = r_scaler.fit_transform(df)

df_r = pd.DataFrame(df_r, columns=col_names)

fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('After RobustScaler')

sns.kdeplot(df_r['beta'], ax=ax1)
sns.kdeplot(df_r['exponential'], ax=ax1)
sns.kdeplot(df_r['normal_p'], ax=ax1)
sns.kdeplot(df_r['normal_l'], ax=ax1)
sns.kdeplot(df_r['bimodal'], ax=ax1)
sns.kdeplot(df_r['normal_big'], ax=ax1);
```

Let's check the minimums and maximums for each column after RobustScaler.

```
df.describe()
```

```
df_r.describe()
```

```
np.mean((df["beta"] - df["beta"].median()) / (df["beta"].quantile(0.75) -
df["beta"].quantile(0.25)))
```

Как и MinMaxScaler, в распределении с большими значениями -normal_big- теперь масштаб похож на другие функции. Обратите внимание, что RobustScaler не масштабирует данные в заранее определенный интервал, как MinMaxScaler.

Диапазон для каждой функции после применения RobustScaler больше, чем для MinMaxScaler.

Используйте RobustScaler, если вы хотите уменьшить влияние выбросов по сравнению с MinMaxScaler.

StandardScaler

StandardScaler стандартизирует каждый признак: вычитает среднее и делит на стандартное отклонение.

На данных, использованных для `fit`, неконстантные признаки получают среднее 0 и дисперсию 1. Форма распределения сохраняется: стандартизация не превращает произвольное распределение в нормальное.

Около 68% значений лежат между −1 и 1 только для нормального распределения, а не для любого стандартизированного признака.

```
s_scaler = preprocessing.StandardScaler()
df_s = s_scaler.fit_transform(df)

df_s = pd.DataFrame(df_s, columns=col_names)

fig, (ax1) = plt.subplots(ncols=1, figsize=(10, 8))
ax1.set_title('After StandardScaler')

sns.kdeplot(df_s['beta'], ax=ax1)
sns.kdeplot(df_s['exponential'], ax=ax1)
sns.kdeplot(df_s['normal_p'], ax=ax1)
sns.kdeplot(df_s['normal_l'], ax=ax1)
sns.kdeplot(df_s['bimodal'], ax=ax1)
sns.kdeplot(df_s['normal_big'], ax=ax1);
```

Qutie a nice chart, don't you think? You can see that all features now have 0 mean.

Let's check the minimums and maximums for each column after StandardScaler.

```
df.describe()
```

```
df_s.describe()
```

```
np.mean((df["beta"] - df["beta"].mean()) / np.std(df["beta"]))
```

На графике признаки имеют среднее, близкое к нулю, и единичную дисперсию. Их диапазоны при этом могут различаться.

StandardScaler часто полезен для регуляризованных линейных моделей, ближайших соседей и PCA. Выбор преобразования проверяют по валидации; заранее лучшего для всех задач варианта нет.

Диапазон для каждой функции примерно как в RobustScaler.

Normalizer

Normalizer нормализует каждую строку — вектор признаков одного объекта. Столбцы не нужно транспонировать.

В этом примере `normal_big` намного больше остальных признаков, поэтому он будет доминировать в норме строки. Нормализация строк не заменяет выбор подходящего масштаба отдельных признаков.

На графике сравним один объект после L2-нормализации без `normal_big` и с ним. В обоих случаях нормируется вектор целиком, а не отдельные столбцы.

По умолчанию нормализация L2 применяется к каждому наблюдению, так что значения в строке имеют единичную норму. Единица нормы с L2 означает, что если каждый элемент возвести в квадрат и просуммировать, сумма будет равна 1. В качестве альтернативы, вместо нормализации L2 может быть применена нормализация L1.

```
df.head()
```

```
n_scaler = preprocessing.Normalizer(norm="l2")
df_n = pd.DataFrame(n_scaler.fit_transform(df), columns=col_names, index=df.index)

ordinary_columns = [column for column in col_names if column != 'normal_big']
first_row = df.index[0]
row_without_big = preprocessing.Normalizer(norm='l2').fit_transform(
    df.loc[[first_row], ordinary_columns]
)[0]

fig, axes = plt.subplots(ncols=2, figsize=(12, 5))
axes[0].bar(ordinary_columns, row_without_big)
axes[0].set_title('One normalized object: without normal_big')
axes[1].bar(col_names, df_n.loc[first_row].to_numpy())
axes[1].set_title('One normalized object: with normal_big')
for axis in axes:
    axis.set_ylim(-1.05, 1.05)
    axis.set_ylabel('Feature value after row-wise L2 normalization')
    axis.tick_params(axis='x', rotation=45)
fig.tight_layout();
```

```
df.describe()
```

```
# L2-норма каждой строки должна равняться 1.
np.linalg.norm(df_n.to_numpy(), axis=1)[:5]
```

```
df_n.describe()
```

Combined Plot

```
# Combined plot.

fig, (ax0, ax1, ax2, ax3) = plt.subplots(ncols=4, figsize=(20, 8))


ax0.set_title('Original Distributions')

sns.kdeplot(df['beta'], ax=ax0)
sns.kdeplot(df['exponential'], ax=ax0)
sns.kdeplot(df['normal_p'], ax=ax0)
sns.kdeplot(df['normal_l'], ax=ax0)
sns.kdeplot(df['bimodal'], ax=ax0)
#sns.kdeplot(df['normal_big'], ax=ax0);


ax1.set_title('After MinMaxScaler')

sns.kdeplot(df_mm['beta'], ax=ax1)
sns.kdeplot(df_mm['exponential'], ax=ax1)
sns.kdeplot(df_mm['normal_p'], ax=ax1)
sns.kdeplot(df_mm['normal_l'], ax=ax1)
sns.kdeplot(df_mm['bimodal'], ax=ax1)
sns.kdeplot(df_mm['normal_big'], ax=ax1);


ax2.set_title('After RobustScaler')

sns.kdeplot(df_r['beta'], ax=ax2)
sns.kdeplot(df_r['exponential'], ax=ax2)
sns.kdeplot(df_r['normal_p'], ax=ax2)
sns.kdeplot(df_r['normal_l'], ax=ax2)
sns.kdeplot(df_r['bimodal'], ax=ax2)
sns.kdeplot(df_r['normal_big'], ax=ax2);


ax3.set_title('After StandardScaler')

sns.kdeplot(df_s['beta'], ax=ax3)
sns.kdeplot(df_s['exponential'], ax=ax3)
sns.kdeplot(df_s['normal_p'], ax=ax3)
sns.kdeplot(df_s['normal_l'], ax=ax3)
sns.kdeplot(df_s['bimodal'], ax=ax3)
sns.kdeplot(df_s['normal_big'], ax=ax3);
```

Все три преобразования изменяют масштаб столбцов. Они сохраняют порядок значений внутри каждого признака, но меняют расстояния в многомерном пространстве, поскольку масштабируют разные признаки по-разному.

Как выбрать преобразование

- `MinMaxScaler` : нужен заданный диапазон на обучающих данных; учитывайте чувствительность к выбросам.
- `RobustScaler` : масштабирование по медиане и межквартильному размаху уменьшает влияние крайних значений на оценку масштаба. Не удаляйте реальные наблюдения автоматически.
- `StandardScaler` : нужны признаки со средним 0 и дисперсией 1. Это не преобразование к нормальному распределению.
- `Normalizer` : нужна единичная L1- или L2-норма каждого объекта; он работает по строкам.

В примерах распределений выше мы изучаем само преобразование, без оценки модели. Далее для оценки качества используем `Pipeline` : преобразование обучается заново внутри каждого train-фолда, а validation-фолд только преобразуется.

Документация: предобработка и утечка данных (https://scikit-learn.org/stable/common_pitfalls.html#data-leakage).

Трансформация значений

```
dataset = pd.read_csv(DATA_DIR / 'train.csv')
```

```
dataset.head().T
```

```
from sklearn.base import clone
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.pipeline import make_pipeline
```

```
from sklearn.model_selection import StratifiedKFold
```

```
X = dataset.drop(columns=['price_range']).to_numpy(dtype=float)
y = dataset['price_range'].to_numpy(dtype=int)
# Не делаем fit_transform на всей выборке перед cross-validation.
```

```
from sklearn.metrics import accuracy_score, mean_absolute_error
```

```
lr = LogisticRegression(solver='newton-cholesky', max_iter=1000)
scaled_lr = make_pipeline(preprocessing.MinMaxScaler(), clone(lr))
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
skf
```

```
ans = []
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model = clone(lr)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    ans.append(accuracy_score(y_test, preds))
```

```
raw_accuracy = float(np.mean(ans))
print(f'Accuracy без масштабирования: {raw_accuracy:.3f}')
```

```
ans = []
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model = clone(scaled_lr)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    ans.append(accuracy_score(y_test, preds))
```

```
scaled_accuracy = float(np.mean(ans))
print(f'Accuracy с MinMaxScaler внутри фолда: {scaled_accuracy:.3f}')
print('Масштабирование меняет и влияние регуляризации; рост accuracy не гарантирован.')
```

```
from sklearn.model_selection import KFold, cross_val_predict
```

```
from sklearn.metrics import confusion_matrix
```

```
y_predict_scaled = cross_val_predict(scaled_lr, X, y, cv=skf, method='predict')
print(f'OOF accuracy с масштабированием: {accuracy_score(y, y_predict_scaled):.3f}')
```

```
y_predict_raw = cross_val_predict(lr, X, y, cv=skf, method='predict')
print(f'OOF accuracy без масштабирования: {accuracy_score(y, y_predict_raw):.3f}')
```

```
cf_matrix = confusion_matrix(y, y_predict_scaled)
```

```
np.sum((y_predict_scaled == 1) & (y == 0))
```

```
np.sum((y_predict_scaled == 1) & (y == 2))
```

```
plt.figure(figsize=(7, 5))
sns.heatmap(cf_matrix, annot=True, fmt='.4g')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.title('OOF confusion matrix: MinMaxScaler + LogisticRegression');
```

Предобработка категориальных признаков

Сравним целочисленные коды (`OrdinalEncoder`) и One-Hot Encoding на задаче предсказания цены пиццы. Для номинальных категорий коды создают искусственный порядок.

Превью кодировок ниже нужны только для иллюстрации. Качество считаем по исходным признакам через `ColumnTransformer` и `Pipeline` : encoder обучается только на train-фолде. Неизвестные категории в validation обрабатываем явно.

Целевая переменная численная, поэтому для обеих моделей используем одинаковый `KFold` , а не `StratifiedKFold` .

```
dataset2 = pd.read_csv(DATA_DIR / 'pizza_v2.csv')
```

```
dataset2.head()
```

```
dataset2['price_rupiah'] = pd.to_numeric(
    dataset2['price_rupiah'].astype(str)
    .str.replace('Rp', '', regex=False)
    .str.replace(',', '', regex=False)
)
```

```
dataset2.head()
```

```
dataset2['diameter'] = pd.to_numeric(
    dataset2['diameter'].astype(str).str.replace(' inch', '', regex=False)
)
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder

categorical_columns = ['company', 'topping', 'variant', 'size',
                       'extra_sauce', 'extra_cheese', 'extra_mushrooms']
numerical_columns = ['diameter']
```

```
dataset2["company"].value_counts()
```

```
label_encoder = OrdinalEncoder()
label_demo = pd.DataFrame(
    label_encoder.fit_transform(dataset2[categorical_columns]),
    columns=categorical_columns, index=dataset2.index,
)
label_demo.head()
```

```
label_demo['company'].value_counts()
```

```
pd.DataFrame({
    'company': dataset2['company'],
    'company_code': label_demo['company'],
}).head()
```

```
dataset2["company"].value_counts()
```

```
X = dataset2.drop(columns=['price_rupiah'])
y = dataset2['price_rupiah'].to_numpy(dtype=float)
```

```
ordinal_preprocessor = ColumnTransformer([
    ('categories', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1),
     categorical_columns),
    ('numbers', 'passthrough', numerical_columns),
])
L = make_pipeline(ordinal_preprocessor, LinearRegression())
kf = KFold(n_splits=10, shuffle=True, random_state=42)
```

```
y_predict_ordinal = cross_val_predict(L, X, y, cv=kf, method='predict')
```

```
dataset2.describe()
```

```
mae_ordinal = mean_absolute_error(y, y_predict_ordinal)
print(f'OOF MAE с кодами категорий: {mae_ordinal:,.0f} рупий')
```

```
# Превью One-Hot Encoding для одного признака; не используем его для CV.
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
```

```
new_ohe_features_company = ohe.fit_transform(dataset2[['company']])
pd.DataFrame(
    new_ohe_features_company,
    columns=ohe.get_feature_names_out(['company']), index=dataset2.index,
).head()
```

```
onehot_preprocessor = ColumnTransformer([
    ('categories', OneHotEncoder(sparse_output=False, handle_unknown='ignore'),
     categorical_columns),
    ('numbers', 'passthrough', numerical_columns),
])
onehot_model = make_pipeline(onehot_preprocessor, LinearRegression())
```

```
y_predict_onehot = cross_val_predict(onehot_model, X, y, cv=kf, method='predict')
```

```
mae_onehot = mean_absolute_error(y, y_predict_onehot)
pd.DataFrame({
    'encoding': ['OrdinalEncoder', 'OneHotEncoder'],
    'OOF MAE, рупий': [mae_ordinal, mae_onehot],
})
```

```
```

```
```